

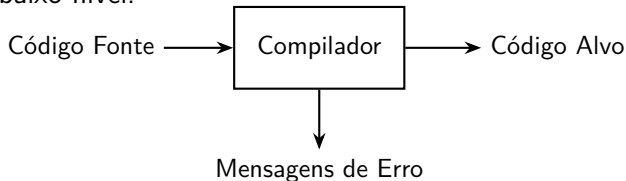
Construção de Compiladores

Fases da Análise e o Analisador Léxico

Compiladores — Livro Base: Aho, Lam, Sethi, Ullman (ALSU) (Livro do Dragão)

Visão Geral de um Compilador

O compilador atua como um tradutor de software de alto nível para uma representação executável ou de baixo nível.



A Estrutura da Construção: Duas Fases

1. **Fase de Análise (Front-End):** Transforma, decompõe e analisa o código fonte em busca de sua estrutura e significado. Compreende as análises linear, hierárquica e semântica, gerando uma representação intermediária.
2. **Fase de Síntese (Back-End):** Constrói o código alvo a partir da representação intermediária através de otimizações e geração de código.

As Três Etapas da Fase de Análise

1. Análise Linear (Análise Léxica)

O código fonte é lido da esquerda para a direita. Os caracteres sequenciais são agrupados em unidades atômicas chamadas **tokens**.

2. Análise Hierárquica (Análise Gramatical ou Sintática)

Os tokens gerados são agrupados de forma estruturada em construções aninhadas (em forma de árvore) guiadas por regras gramaticais.

3. Análise Semântica

Realiza verificações não desenvolvidas pelas outras análises para garantir a consistência de significado do programa (por exemplo, a verificação de tipos e a validação de parâmetros de funções).

Exemplo Pequeno e Prático - Fase de Análise

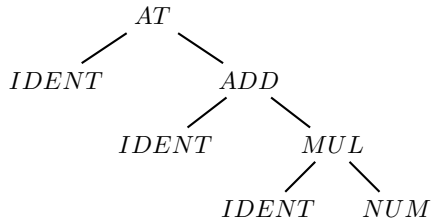
Considere a seguinte atribuição: $\text{montante} = \text{depósito} + \text{taxa} \times 60$

Resultado da Análise Léxica (Tokens):

IDENT AT IDENT ADD IDENT MUL NUM

Sendo: *IDENT* $\equiv$ Identificador, *AT* $\equiv$ Atribuição, *ADD* $\equiv$ Adição, *MUL* $\equiv$ Multiplicação, *NUM* $\equiv$ Número.

Resultado da Análise Hierárquica (Árvore de Derivação):



Resultado da Análise Semântica: As operações estão bem definidas.

A Tabela de Símbolos e Detecção de Erros

Tabela de Símbolos

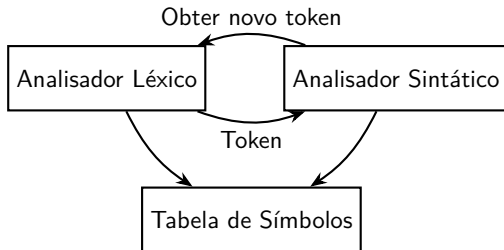
É uma estrutura de dados central contendo uma entrada para cada identificador descoberto no código fonte. Armazena os atributos essenciais associados a ele (por exemplo: escopo, tipo e número de parâmetros).

Tipos de Erros - Fase de Análise

- ▶ **Erro Léxico:** Ocorre quando uma cadeia de caracteres não pode ser mapeada para nenhum token válido da linguagem (Exemplo: `@abc` ou um caracter inválido no meio de um identificador).
- ▶ **Erro Sintático:** Ocorre quando a sequência de tokens viola as regras estruturais da gramática (Exemplo: $a + \times b$).
- ▶ **Erro Semântico:** Ocorre, por exemplo, quando uma operação legítima sintaticamente é aplicada sobre tipos incompatíveis da linguagem.

Funcionamento do Analisador Léxico

O analisador léxico atua sob demanda do analisador sintático, gerenciando o fluxo com a tabela de símbolos:



Funcionamento do Analisador Léxico

Diferença de Conceitos Fundamentais

- ▶ **Lexema:** É a sequência concreta de caracteres lida do código fonte.
Exemplo: `const`, `pi`, `3.1416`
- ▶ **Token:** É a unidade atômica abstrata à qual o lexema pertence.
Exemplo: `const` → token `CONST`, `pi` → token `IDENT`, `3.1416` → token `NUM`

Importante: Os tokens estão relacionados aos símbolos terminais de uma gramática.

Categorias Gerais de Tokens

Em geral, costuma-se classificar os tokens em cinco categorias fundamentais:

Grupos Principais de Tokens

1. **Palavras Reservadas:** Um token específico mapeado para cada palavra reservada da linguagem (Exemplo: IF, THEN, ELSE, CONST).
2. **Operadores:** Um token próprio para operadores, podendo ser tratados individualmente ou agrupados por classe (Exemplo: aritméticos, relacionais).
3. **Identificadores:** Um token único representando todas as variáveis, funções ou classes criadas (Exemplo: IDENT representando *montante*, *pi*, *soma*).

Categorias Gerais de Tokens

Grupos Principais de Tokens

4. **Constantes Literais:** Um ou mais tokens representando valores numéricos e strings explícitas no código (Exemplo: NUM para 3.1416 ou 60).
5. **Símbolos de Pontuação:** Um token dedicado para cada delimitador estrutural ou pontuação (Exemplo: abre/fecha parênteses (AP, FP), ponto e vírgula (PV), vírgulas (V)).

Especificação de Tokens

Mecanismo de Definição

As ferramentas formais utilizadas para realizar a especificação completa de cada um desses padrões de tokens no projeto do compilador são as **Expressões Regulares (ER)** ou **Diagramas de Transição**.

Nota: Uma vez especificados, esses tokens passam a ser tratados estritamente como os símbolos terminais de uma gramática.

Definições Regulares

Para simplificar a especificação de padrões de tokens, podemos criar apelidos (ou macros) que representam expressões regulares menores e reutilizáveis.

Definição Formal

Uma definição regular é uma sequência de atribuições que assume a seguinte estrutura hierárquica:

$$d_1 \rightarrow R_1$$

$$d_2 \rightarrow R_2$$

$$\vdots$$

$$d_n \rightarrow R_n$$

Restrições e Regras de Formação

- ▶ Cada d_i é um **nome distinto** (uma macro única).
- ▶ Cada R_i é uma expressão regular definida sobre o alfabeto expandido:

$$\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}.$$

Definições Regulares: Identificadores em Pascal

Para ilustrar a aplicação prática das definições regulares, analisamos como o compilador do Pascal especifica a formação de um identificador:

Especificação: Um identificador obrigatoriamente precisa começar com uma letra. Após a primeira letra, é permitido concatenar zero ou mais letras ou dígitos.

Definições Regulares: Identificadores em Pascal

Para ilustrar a aplicação prática das definições regulares, analisamos como o compilador do Pascal especifica a formação de um identificador:

Especificação: Um identificador obrigatoriamente precisa começar com uma letra. Após a primeira letra, é permitido concatenar zero ou mais letras ou dígitos.

Especificação do Token IDENT

Utilizando macros para mapear letras e dígitos sequencialmente:

$$\text{letra} \rightarrow A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z$$
$$\text{dígito} \rightarrow 0 \mid 1 \mid \dots \mid 9$$
$$\text{id} \rightarrow \text{letra} (\text{letra} \mid \text{dígito})^*$$

Definições Regulares: Números em Pascal

A especificação de constantes numéricas reais e de ponto flutuante em Pascal exige a composição de partes opcionais (como frações e expoentes):

Especificação: Um número precisa ter pelo menos um dígito. Ele pode conter uma parte fracionária e também pode ser escrito na notação científica.

Definições Regulares: Números em Pascal

A especificação de constantes numéricas reais e de ponto flutuante em Pascal exige a composição de partes opcionais (como frações e expoentes):

Especificação: Um número precisa ter pelo menos um dígito. Ele pode conter uma parte fracionária e também pode ser escrito na notação científica.

Especificação do Token NUM

$$\text{dígitos} \rightarrow \text{dígito} \text{dígitos}^*$$
$$\text{fração-opcional} \rightarrow . \text{dígitos} \mid \varepsilon$$
$$\text{expoente-opcional} \rightarrow (E (+ \mid - \mid \varepsilon) \text{dígitos}) \mid \varepsilon$$
$$\text{num} \rightarrow \text{dígitos} \text{fração-opcional} \text{expoente-opcional}$$

Diagrama de Transição: Operadores Relacionais (Relop)

A construção de um analisador léxico pode ser baseada em diagramas de transição. Neste caso, o analisador léxico usa tais diagramas para reconhecer os padrões dos tokens. Veja o exemplo para os operadores relacionais ($<$, $\leq$, $=$, $\neq$, $>$, $\geq$):

Diagrama de Transição: Operadores Relacionais (Relop)

A construção de um analisador léxico pode ser baseada em diagramas de transição. Neste caso, o analisador léxico usa tais diagramas para reconhecer os padrões dos tokens. Veja o exemplo para os operadores relacionais ($<$, $\leq$, $=$, $\neq$, $>$, $\geq$):

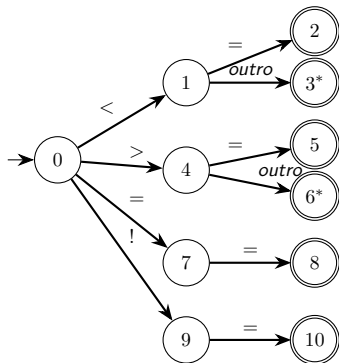


Diagrama de Transição: Operadores Relacionais (Relop)

- ▶ **Nota 1:** O caracter asterisco (*) indica um estado onde ocorreu o “*lookahead*” (um caracter a mais foi lido e deve ser devolvido ao fluxo para nova consideração).
- ▶ **Nota 2:** A execução sobre os diagramas param ao chegar em um estado final.

O Mecanismo de *Lookahead* (Olhar Adiante)

Na análise léxica, o **lookahead** é a ação de ler caracteres adicionais para certificar-se do fim de um padrão.

Como o Buffer da Entrada Gerencia o Fluxo

- ▶ **A Necessidade:** Para reconhecer o token correspondente ao operador $<$, o analisador léxico é obrigado a ler o caractere seguinte. Se o caractere seguinte for uma letra (ex: x), o analisador descobre que o operador acabou.
- ▶ **A Devolução:** Como a letra x não pertence ao operador relacional, ela é reintroduzida no fluxo de entrada (o ponteiro do buffer retrocede uma posição).
- ▶ **O Próximo Token:** Na chamada seguinte, o processo de varredura começará exatamente a partir daquela letra x devolvida.

Notação Gráfica nos Diagramas: Nos diagramas de transição, todo estado final que exige a devolução de um caracter de *lookahead* é marcado com um *asterisco* (*).

Diagramas de Transição: Identificadores e Brancos

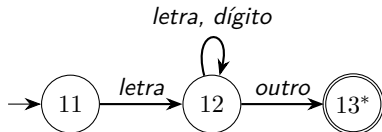
Mapeamento dos diagramas para o reconhecimento de nomes genéricos e eliminação de caracteres vazios de formatação:

Reconhecimento de Identificadores

Diagramas de Transição: Identificadores e Brancos

Mapeamento dos diagramas para o reconhecimento de nomes genéricos e eliminação de caracteres vazios de formatação:

Reconhecimento de Identificadores

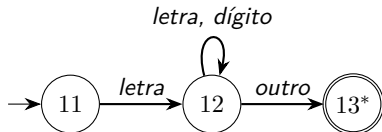


Espaços em Branco

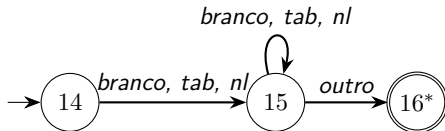
Diagramas de Transição: Identificadores e Brancos

Mapeamento dos diagramas para o reconhecimento de nomes genéricos e eliminação de caracteres vazios de formatação:

Reconhecimento de Identificadores



Espaços em Branco

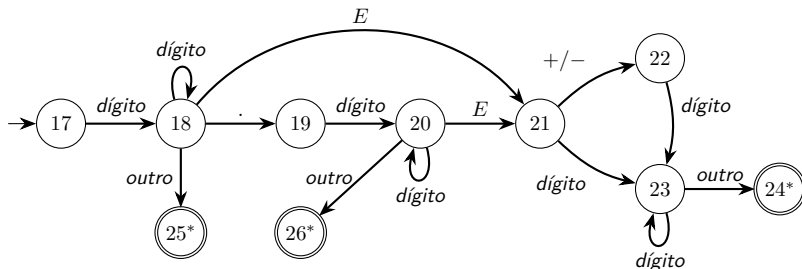


Passagem de Atributos de Tokens

Quando múltiplos lexemas casam com o padrão de token (Exemplo: o token NUM pode casar com os lexemas concreto 0.1, 0 ou 333), o analisador léxico repassa, por exemplo, o lexema lido como um **atributo** para o analisador sintático.

Diagrama de Transição: Números

Abaixo apresenta-se o diagrama para reconhecimento estrutural de números inteiros, reais e com notação científica em ponto flutuante:



- Os estados 24*, 25* e 26* finalizam a aceitação e acionam o retrocesso de um caractere no buffer lido.

Arquitetura do Analisador Léxico Baseado em Diagramas

A implementação de um analisador léxico prático mapeia os diagramas de transição diretamente para blocos de código estruturados.

Estratégia de Implementação

- ▶ **Gerenciamento de Diagramas:** Temos vários diagramas de transição, testados em uma ordem sequencial.
- ▶ **Controle de Estados:** Uma variável global *state* armazena o número do estado atual dentro do diagrama em execução.
- ▶ **Desvio de Fluxo:** Utiliza-se uma estrutura de seleção múltipla (comando `switch`) baseada no valor de *state* para executar as tarefas e ler os caracteres de entrada.

Analizador Léxico - Diagrama de Transição - *Relop*

```
TOKEN getRelop() {
    state = startStateRelop();
    TOKEN retToken = new(RELOP);
    while (1) {
        switch (state) {
            case 0:
                c = nextChar();
                if (c == '<') state = 1;
                else if (c == '=') state = 7;
                else if (c == '>') state = 4;
                else if (c == '!') state = 9;
                else fail(); /* Não é um operador relacional. */
            case 1: ... /* Todos os outros casos são tratados. Destaque para o 6 abaixo. */
            case 6: retract(); retToken.attribute = GT; return(retToken);
        }
    }
}
```

Tratamento de Falhas

Quando um diagrama falha em reconhecer um padrão a partir do estado inicial, o buffer e os ponteiros precisam ser restaurados de forma consistente.

O Mecanismo da Função `fail()`

No estado inicial $state = 0$, se o próximo caractere lido não puder iniciar o token atual (RELOP):

- ▶ A rotina `fail()` é chamada imediatamente.
- ▶ O ponteiro de leitura do buffer retrocede para a **posição original de entrada**.
- ▶ A variável `state` é atualizada com o estado inicial do **próximo diagrama** a ser testado.
- ▶ Se não houver mais diagramas disponíveis, a fase de gerenciamento de erros léxicos assume o controle da entrada.

Como Implementar um Analisador Léxico Baseado em Diagramas de Transição?

Estratégias de Implementação

Abordagem Serial: Executa um diagrama por vez. A função `fail()` reinicia o ponteiro e configura *state* para o próximo diagrama. Permite diagramas para palavras reservadas (como THEN), mas exige ordenação estrita entre eles.

Abordagem Paralela: Alimenta todos os diagramas simultaneamente com o mesmo caracter. Se dois ou mais tokens forem reconhecidos (conflito), o compilador adota a regra do *longest match* (ex: reconhecer THENEXT como um único token IDENT ao invés da palavra reservada THEN).

Abordagem Combinada: Une todos os diagramas de transição independentes em um único grande autômato finito unificado.

Precisamos de Diagramas para Palavras Reservadas?

Estratégia de Fusão de Diagramas

Em vez de criar diagramas triviais e separados para cada palavra reservada, o analisador léxico pode implementar apenas **um único diagrama de transição genérico**: o diagrama para o token IDENT.

Resolução de Conflitos

Quando um lexema de texto (como `if` ou `then`) é isolado pelo diagrama de identificadores:

1. O analisador léxico faz uma consulta em uma tabela *hash* interna.
2. **Palavra Reservada Encontrada:** Se o lexema já residir na tabela, o analisador converte a classificação e retorna o token correspondente (Ex: IF, THEN, ELSE).
3. **Identificador Puro:** Se o lexema não for encontrado na tabela, ele segue seu fluxo padrão e um token IDENT é devolvido.

Diagramas para Palavras Reservadas?

Vantagens Arquiteturais Dessa Abordagem

- ▶ Elimina a necessidade de ordenação estrita na implementação serial.
- ▶ Otimiza o uso da memória de estados.
- ▶ Acelera a varredura do código fonte através de consultas de tempo $O(1)$.